

Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	RI. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	02
Título de la Práctica	Implementación de acceso secuencial y aleatorio a archivos con métricas de I/O
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 8 de octubre 2026 - viernes 9 de octubre 2026.
Horario	jueves 8 de 08:30 – 09:30 viernes 9 de 11:30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar algoritmos de acceso secuencial para la persistencia masiva de datos en disco mediante el módulo nativo de I/O en JavaScript.

Cuantificar empíricamente la latencia computacional (tiempo de respuesta $T(n)$) y el impacto en la memoria Heap ($S(n)$) al procesar archivos de forma síncrona (bloqueante) versus flujos asíncronos (*Streams*).

Evaluar las decisiones de diseño físico en el acceso a almacenamiento mecánico o de estado sólido bajo los principios de *Green Computing*, analizando cómo la optimización de los accesos I/O reduce el consumo energético del hardware.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial de Node.js (módulo fs).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).
- Recursos Digitales: Sílabo de la asignatura, repositorio base en GitHub.
- Plataforma EVA UNL.

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos (recomendado 4).
- Memoria RAM: Mínimo 4 GB.
- Almacenamiento: Disco HDD o SSD con al menos 1 GB de espacio libre temporal.

5. Procedimiento / Metodología.

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El equipo satelital del APE 001 ahora necesita leer el millón de coordenadas ($N = 10^6$) que han sido transmitidas y almacenadas en un archivo físico en el servidor. El problema radica en que leer el archivo de una sola vez colapsa la memoria RAM del contenedor en la nube. Se debe diseñar una estrategia para leer los datos con el mínimo impacto temporal y espacial.
- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo interactúa el *Event Loop* de JavaScript (motor V8 y libuv) con las llamadas del sistema operativo (Syscalls) para el acceso a disco.
- **Modalidad:** Individual y análisis comparativo grupal posterior.



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Tarea 1: Generación del Conjunto de Datos Masivo (Mock Data)

Para poder medir el I/O, primero necesitamos un archivo físico considerable.

1. En el directorio de su proyecto, cree un archivo llamado `lab02_generador.js`.
2. Implemente el siguiente código para escribir un archivo CSV con 10^6 registros utilizando escritura en flujo (Stream) para no saturar la memoria durante su creación.

```
const fs = require('fs');
const { performance } = require('perf_hooks');

const FILE_NAME = 'coordenadas_masivas.csv';
const N = 1000000;

console.log('[Generador] Iniciando escritura de ${N} registros...');
const start = performance.now();

// Utilizamos un Stream de escritura (Write Stream)
const writableStream = fs.createWriteStream(FILE_NAME);
writableStream.write('id,latitud,longitud\n'); // Cabecera

for (let i = 0; i < N; i++) {
  const lat = (Math.random() * 180 - 90).toFixed(6);
  const lng = (Math.random() * 360 - 180).toFixed(6);
  // Buffer string format
  writableStream.write(`${i},${lat},${lng}\n`);
}

writableStream.end();

writableStream.on('finish', () => {
  const end = performance.now();
  console.log('[Generador] Archivo creado en ${{(end - start) / 1000}.toFixed(2)}
```

```
segundos.`);
```

```
    // Calcular tamaño del archivo en disco
    const stats = fs.statSync(FILE_NAME);
    const sizeInMB = stats.size / (1024 * 1024);
    console.log(`[Generador] Peso físico en disco: ${sizeInMB.toFixed(2)} MB`);
  });
```

3. Ejecute en terminal: `node lab02_generador.js` y verifique la creación del archivo `coordenadas_masivas.csv`.

Tarea 2: Enfoque 1 - Lectura Síncrona (Bloqueante / $O(n)$ Espacial)

Este enfoque representa la manera más intuitiva, pero menos eficiente, de procesar un archivo físico.

1. Cree un archivo llamado `lab02_lectura_sync.js`.
2. Implemente el código para cargar todo el archivo en la RAM.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura Síncrona] Iniciando carga en memoria...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
// I/O Bloqueante: El hilo principal se detiene hasta cargar todo
const data = fs.readFileSync(FILE_NAME, 'utf-8');
// Simulamos procesamiento separando por saltos de línea
const lineas = data.split('\n');
const end = performance.now();
const memoryAfter = process.memoryUsage().heapUsed;
console.log(`[Lectura Síncrona] Total registros leídos: ${lineas.length - 1}`);
console.log(`[Lectura Síncrona] Tiempo de I/O + Parsing: $${((end - start) / 1000).toFixed(2)} segundos.`);
console.log(`[Lectura Síncrona] Consumo Neto de RAM: $${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
```

3. Ejecute node lab02_lectura_sync.js y tabule el tiempo y el impacto en memoria.

Tarea 3: Enfoque 2 - Lectura Asíncrona con Streams ($O(1)$ Espacial)

1. Cree el archivo lab02_lectura_stream.js.
2. Implemente la abstracción de flujos (Streams) para leer fragmentos (*chunks*) secuenciales, reduciendo la complejidad espacial teórica de $S(n) = n \cdot \text{bytes}$ a un constante $S(n) = O(1)$ correspondiente al tamaño del *buffer*.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log('[Lectura por Stream] Iniciando procesamiento...');
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
let totalRegistros = 0;
// Creamos un flujo de lectura secuencial no bloqueante
const readableStream = fs.createReadStream(FILE_NAME, { encoding: 'utf-8' });
readableStream.on('data', (chunk) => {
    // Procesamos el chunk de datos actual sin almacenarlo completo
    // Contamos cuántos saltos de línea hay en este fragmento
    let lineBreakCount = (chunk.match(/\n/g) || []).length;
    totalRegistros += lineBreakCount;
});
readableStream.on('end', () => {
    const end = performance.now();
    const memoryAfter = process.memoryUsage().heapUsed;
    console.log('[Lectura por Stream] Total registros procesados: ${totalRegistros}');
    console.log('[Lectura por Stream] Tiempo de I/O parcializado: $(((end - start) / 1000).toFixed(2)) segundos.');
```

```
    console.log('[Lectura por Stream] Consumo Neto de RAM: $(((memoryAfter -
```

```
memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
```

```
});
```

3. Ejecute `node lab02_lectura_stream.js`. Compare drásticamente la métrica del consumo neto de RAM entre la Tarea 2 y la Tarea 3.

Tarea 4: Análisis Matemático de Rendimiento (ABI)

Basado en los resultados empíricos, los estudiantes investigarán el diseño del bus de datos y el *Thread Pool* interno (libuv) que gestiona estas llamadas.

- Demuestre teóricamente (utilizando notación de orden asintótico) por qué la función bloqueante impone $S(n) = O(n)$ frente al enfoque *Stream* $S(n) = O(1)$.
- Estructure su respuesta relacionándola con el uso energético del servidor (menos RAM implica menor recolección de basura y menor gasto térmico del procesador).

6. Resultados esperados:

- Se obtendrán métricas directas que evidencian cómo una operación aparentemente trivial de lectura/escritura puede causar cuellos de botella de I/O (*Input/Output Bottleneck*). El uso de *Streams* demostrará que la persistencia masiva puede gestionarse secuencialmente manteniendo una huella de memoria plana, logrando un código ambiental y computacionalmente sostenible.

7. Preguntas de Control:

- Desde una perspectiva filosófica profunda y analítica, responda:
 1. **Causa Material:** Al realizar una lectura síncrona mediante `readFileSync`, ¿cuál es la "materia" que se satura en el ordenador y cómo se diferencia físicamente del estado de reposo de un archivo almacenado magnética o electrónicamente en el disco duro?
 2. **Causa Formal:** El archivo CSV (Valores Separados por Comas) posee una "forma" específica (texto secuencial lineal). ¿De qué manera esta forma impone restricciones algorítmicas sobre nuestra capacidad para acceder aleatoriamente a la coordenada $N = 500,000$ sin tener que recorrer las $499,999$ coordenadas previas?
 3. **Causa Eficiente:** Durante el uso de *Streams* asíncronos en Node.js, el *Event Loop* delega la operación al sistema operativo. En este contexto,

¿quién actúa como el verdadero motor (causa eficiente) que transfiere los electrones desde el disco hasta la memoria RAM en fragmentos (chunks)?

4. **Causa Final:** Si todo dato en ejecución debe residir en la memoria principal para ser computado por la CPU, ¿cuál es el propósito (telos) de la "persistencia" en el disco físico, sabiendo que las operaciones de I/O añaden un costo de tiempo monumental en la escala de microsegundos (μs) frente a los nanosegundos (ns) de la memoria caché?

5. **Esencia y Existencia:** Un conjunto de 10^6 coordenadas existe como un bloque monolítico en el archivo data.csv, pero en el enfoque de *Streams*, solo una porción existe en la RAM a la vez. ¿Es conceptualmente el "mismo" conjunto de datos procesado, si en la memoria jamás llegó a existir de manera unificada? Argumente.

8. Evaluación

Criterio Evaluado	Puntaje Máximo	Evidencia Esperada
1. Generación de Data	2 puntos	Captura de consola que demuestre la creación exitosa del archivo de ~ y su peso impreso.
2. Implementación de I/O Bloqueante	2 puntos	Captura de consola ejecutando lab02_lectura_sync.js, mostrando el alto impacto en el consumo de RAM.
3. Implementación de I/O por Streams	2 puntos	Captura ejecutando lab02_lectura_stream.js evidenciando el consumo neto de RAM cercano a 0 o muy bajo.
4. Respuestas Aristotélicas y ABI	3 puntos	Informe en PDF detallando las respuestas de control y el



		análisis matemático $S(n)$ vs $O(1)$.
5. Sustentabilidad Computacional	1 punto	Código comentado indicando la zona exacta donde se ahorran ciclos del Garbage Collector.
Total	10 puntos	

9. Bibliografía

- P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. [Online]. Available: <https://open.palni.org/>
- Node.js Foundation, "File system | Node.js v20.x Documentation", *Node.js API*, 2023. [Online]. Available: <https://nodejs.org/docs/latest-v20.x/api/fs.html>
- S. Tilkov et al., *Node.js in Action*, 2nd ed. Manning Publications, 2017.

10. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén Docente	 Validar únicamente en FirmaEC. Firmado electrónicamente por: CRISTIAN RAMIRO NARVAEZ GUILLEN
Aprobado por	Edison L Coronel Romero Director de Carrera	